Solution: Longest Common Substring

Problem Statement

Given two strings 's1' and 's2', find the length of the longest substring which is common in both the strings.

Example 1:

```
Input: s1 = "abdca"
       s2 = "cbda"
Output: 2
Explanation: The longest common substring is "bd".
```

Example 2:

```
Input: s1 = "passport"
       s2 = "ppsspt"
Output: 3
Explanation: The longest common substring is "ssp".
```

Basic Solution

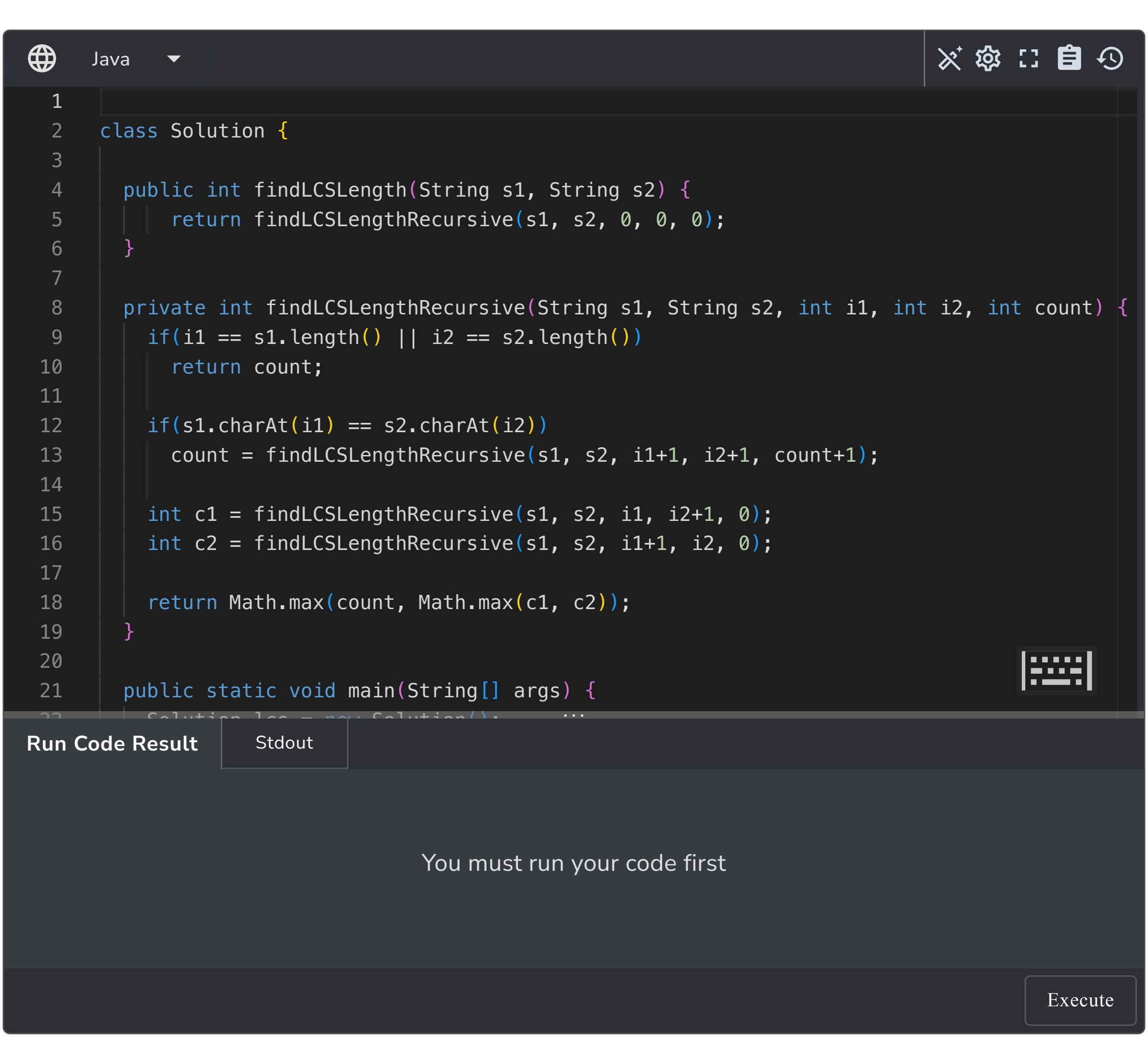
A basic brute-force solution could be to try all substrings of 's1' and 's2' to find the longest common one. We can start matching both the strings one character at a time, so we have two options at any step:

1. If the strings have a matching character, we can recursively match for the remaining lengths and keep a track of the current matching length.
2. If the strings don't match, we start two new recursive calls by skipping one character separately from each string and reset the matching length.

The length of the Longest Common Substring (LCS) will be the maximum number returned by the three recurse calls in the above two options.

Code

Here is the code:



```java

class Solution {

  public int findLCSLength(String s1, String s2) {
      return findLCSLengthRecursive(s1, s2, 0, 0, 0);
  }

  private int findLCSLengthRecursive(String s1, String s2, int i1, int i2, int count) {
    if(i1 == s1.length() || i2 == s2.length())
      return count;

    if(s1.charAt(i1) == s2.charAt(i2))
      count = findLCSLengthRecursive(s1, s2, i1+1, i2+1, count+1);

    int c1 = findLCSLengthRecursive(s1, s2, i1, i2+1, 0);
    int c2 = findLCSLengthRecursive(s1, s2, i1+1, i2, 0);

    return Math.max(count, Math.max(c1, c2));
  }

  public static void main(String[] args) {
```

Because of the three recursive calls, the time complexity of the above algorithm is exponential $O(3^{m+n})$, where 'm' and 'n' are the lengths of the two input strings. The space complexity is $O(m+n)$, this space will be used to store the recursion stack.

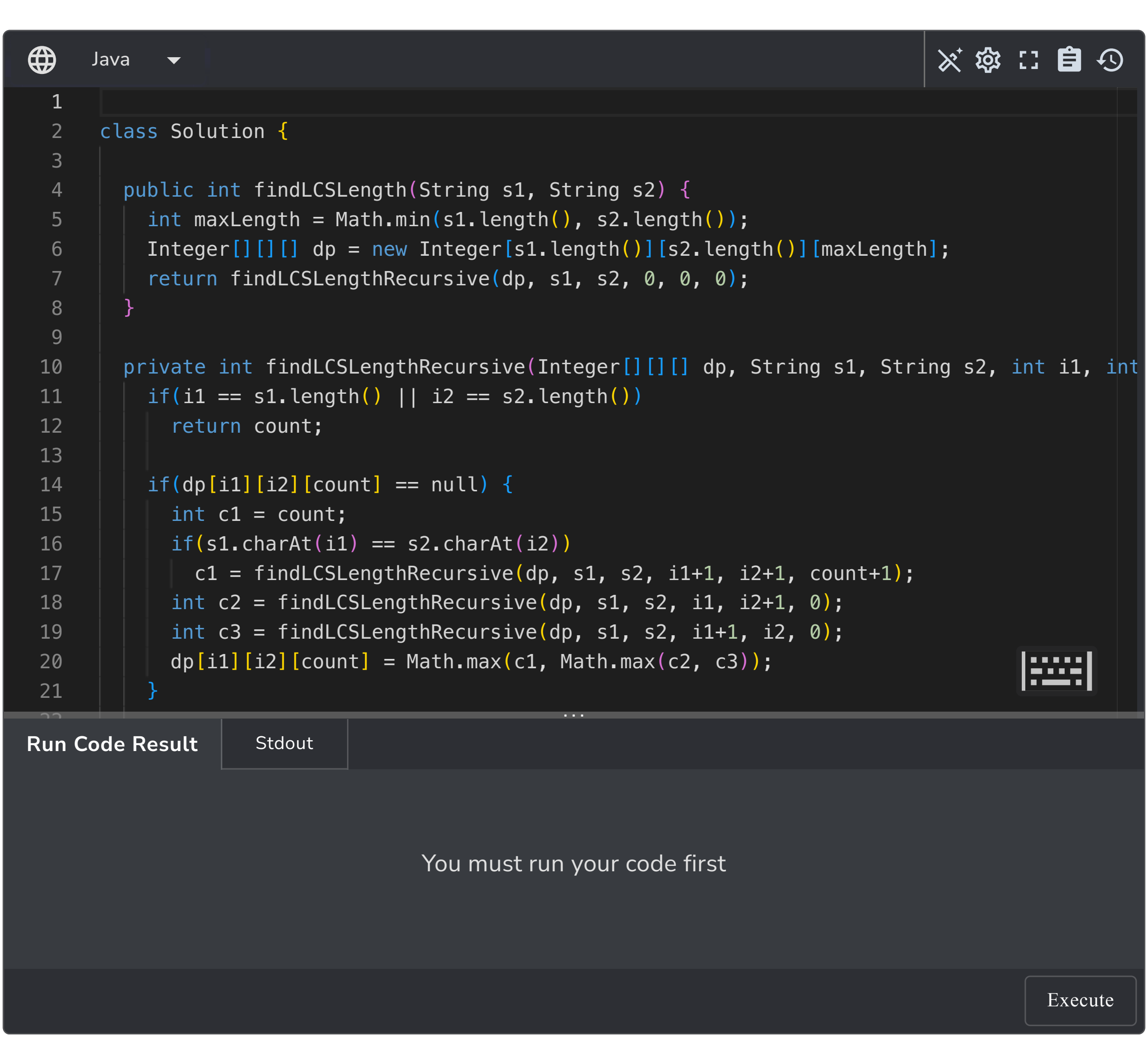
Top-down Dynamic Programming with Memoization

We can use an array to store the already solved subproblems.

The three changing values to our recursive function are the two indexes (i1 and i2) and the 'count'. Therefore, we can store the results of all subproblems in a three-dimensional array. (Another alternative could be to use a hash-table whose key would be a string (i1 + "|" i2 + "|" + count)).

Code

Here is the code:



```java

class Solution {

  public int findLCSLength(String s1, String s2) {
    int maxLength = Math.min(s1.length(), s2.length());
    Integer[][][] dp = new Integer[s1.length()][s2.length()][maxLength];
    return findLCSLengthRecursive(dp, s1, s2, 0, 0, 0);
  }

  private int findLCSLengthRecursive(Integer[][][] dp, String s1, String s2, int i1, int
    if(i1 == s1.length() || i2 == s2.length())
      return count;

    if(dp[i1][i2][count] == null) {
      int c1 = count;
      if(s1.charAt(i1) == s2.charAt(i2))
        c1 = findLCSLengthRecursive(dp, s1, s2, i1+1, i2+1, count+1);
      int c2 = findLCSLengthRecursive(dp, s1, s2, i1, i2+1, 0);
      int c3 = findLCSLengthRecursive(dp, s1, s2, i1+1, i2, 0);
      dp[i1][i2][count] = Math.max(c1, Math.max(c2, c3));
    }
```

Bottom-up Dynamic Programming

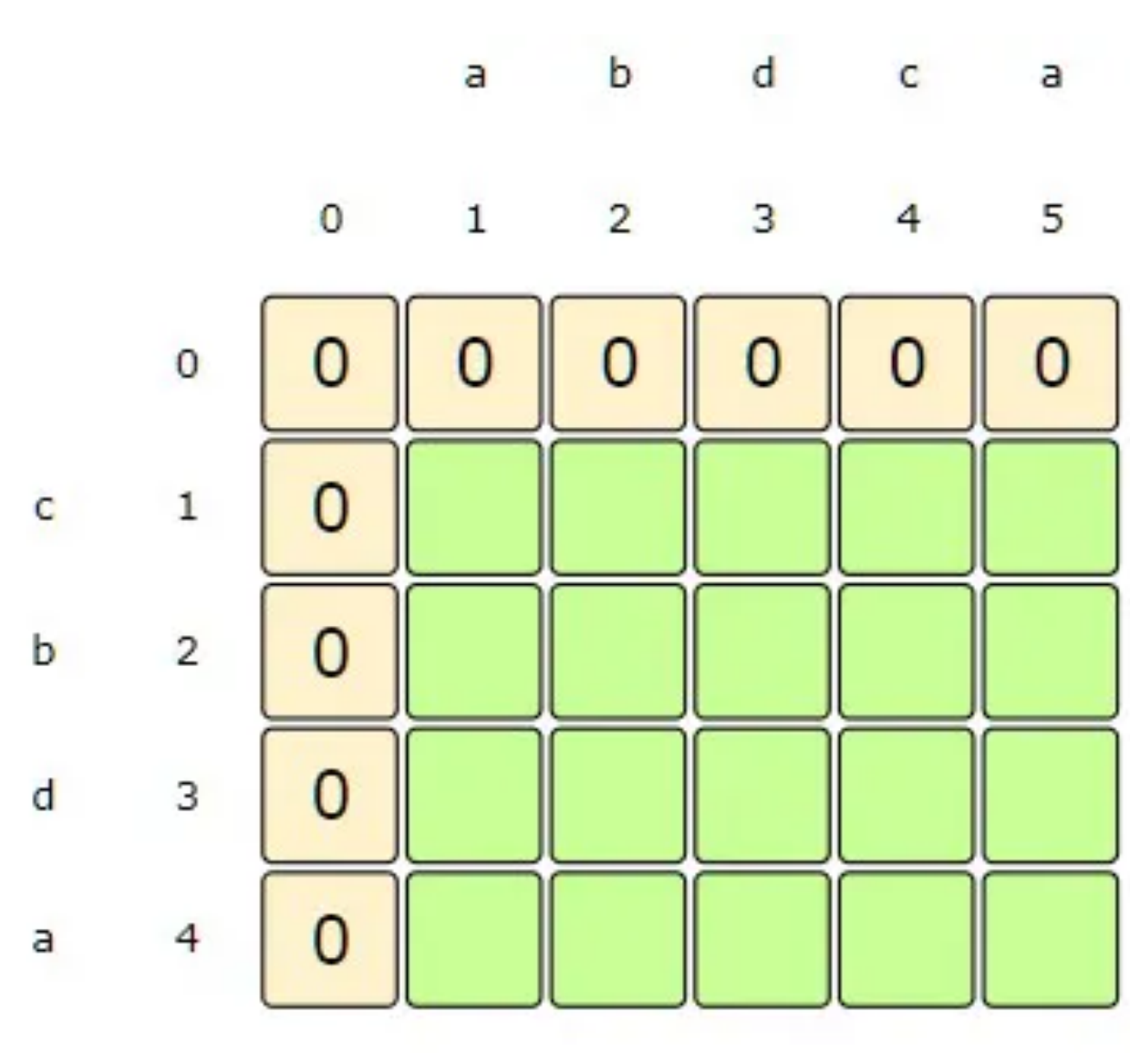
Since we want to match all the substrings of the given two strings, we can use a two-dimensional array to store our results. The lengths of the two strings will define the size of the two dimensions of the array. So for every index 'i' in string 's1' and 'j' in string 's2', we have two options:

1. If the character at `s1[i]` matches `s2[j]`, the length of the common substring would be one plus the length of the common substring till `i-1` and `j-1` indexes in the two strings.
2. If the character at the `s1[i]` does not match `s2[j]`, we don't have any common substring.

So our recursive formula would be:

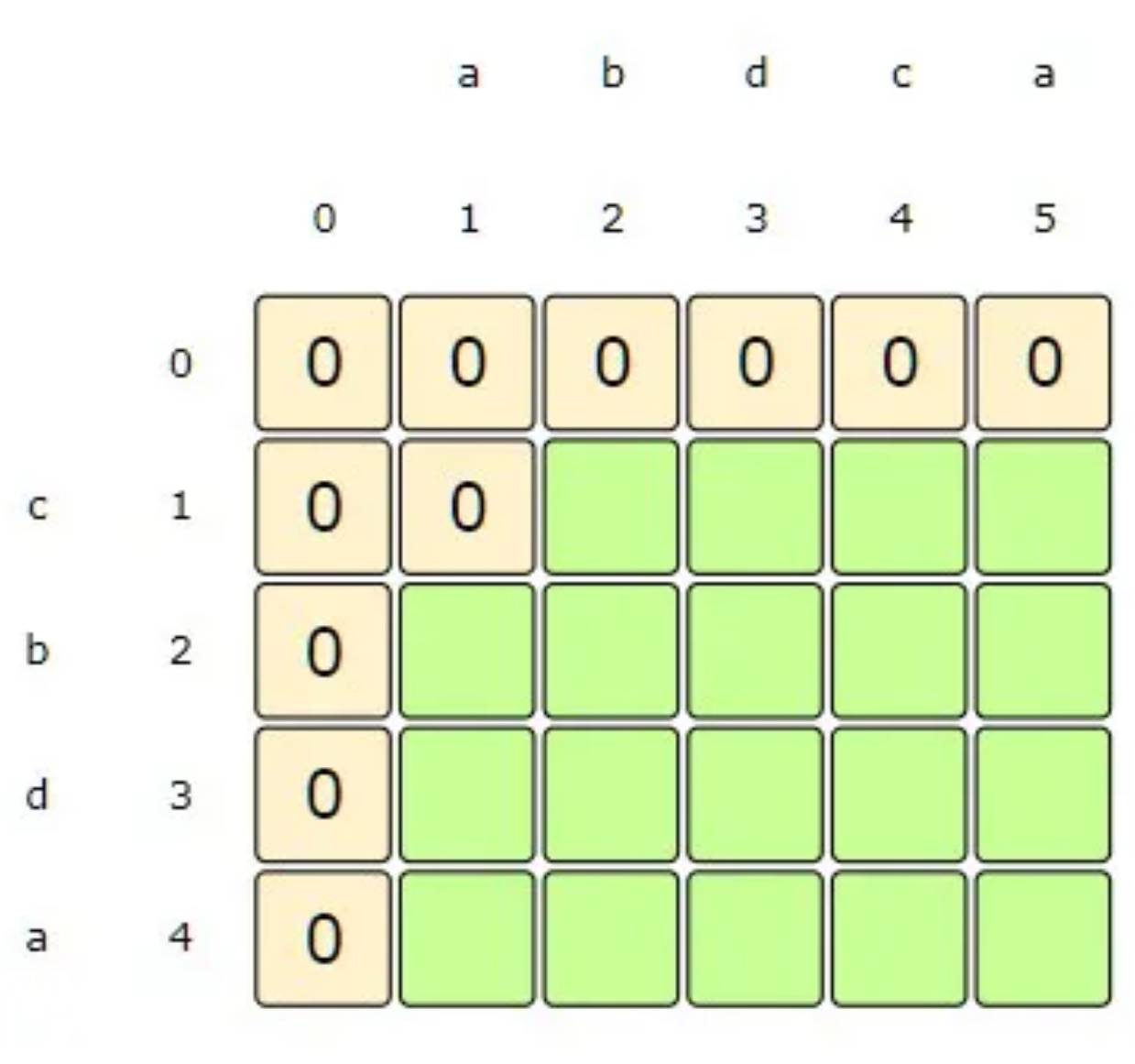
```
if s1[i] == s2[j]
  dp[i][j] = 1 + dp[i-1][j-1]
else
  dp[i][j] = 0
```

Let's draw this visually for "abcda" and "cbda". Starting with a substring of zero lengths, if any one of the string has zero length, then the common substring will be of zero length:



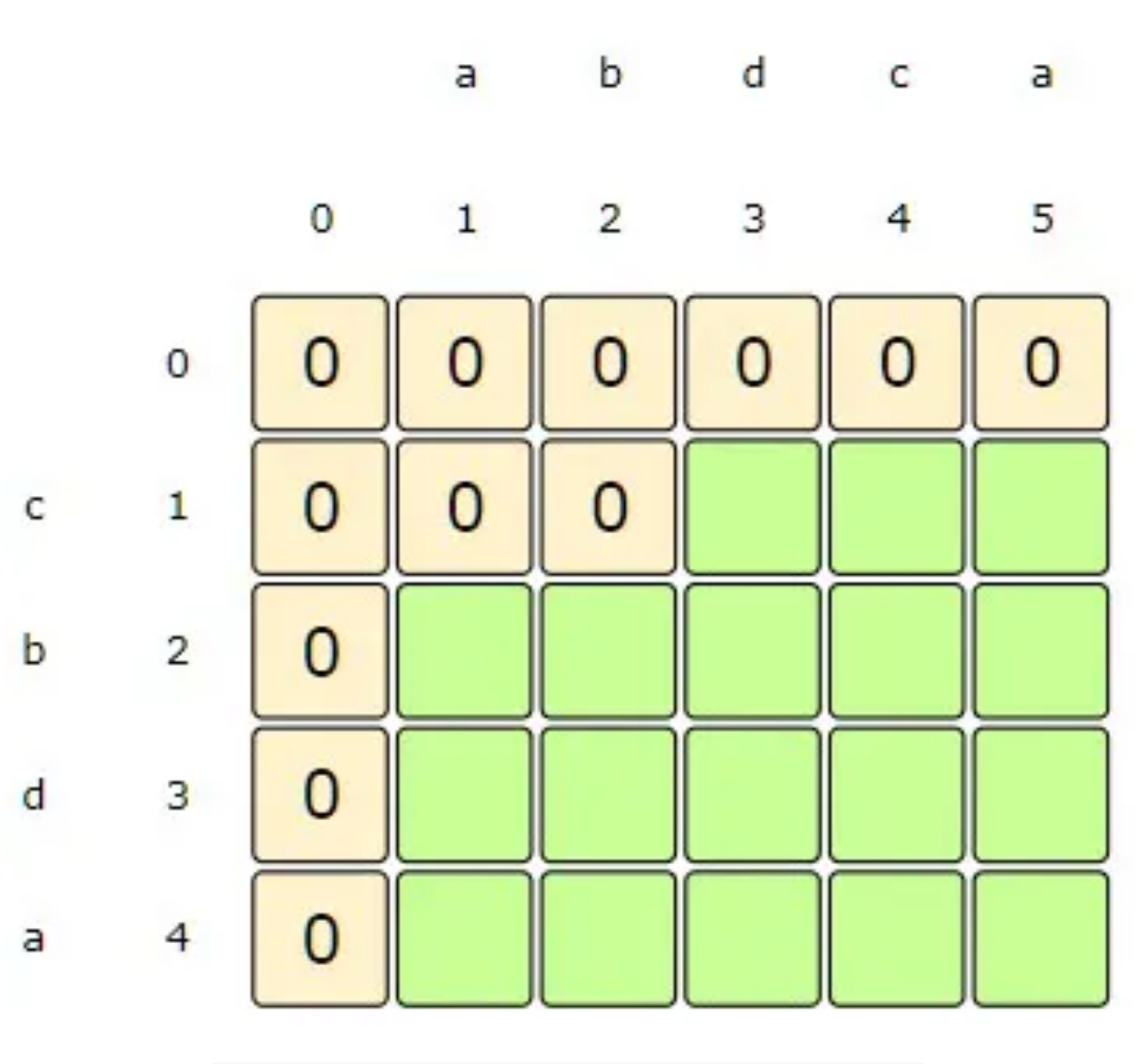
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0					
b	2	0					
d	3	0					
a	4	0					

i:0, j:0-5 and i:0-4, j:0 => dp[i][j] = 0, as we don't have any common substring when one of the string is of zero length



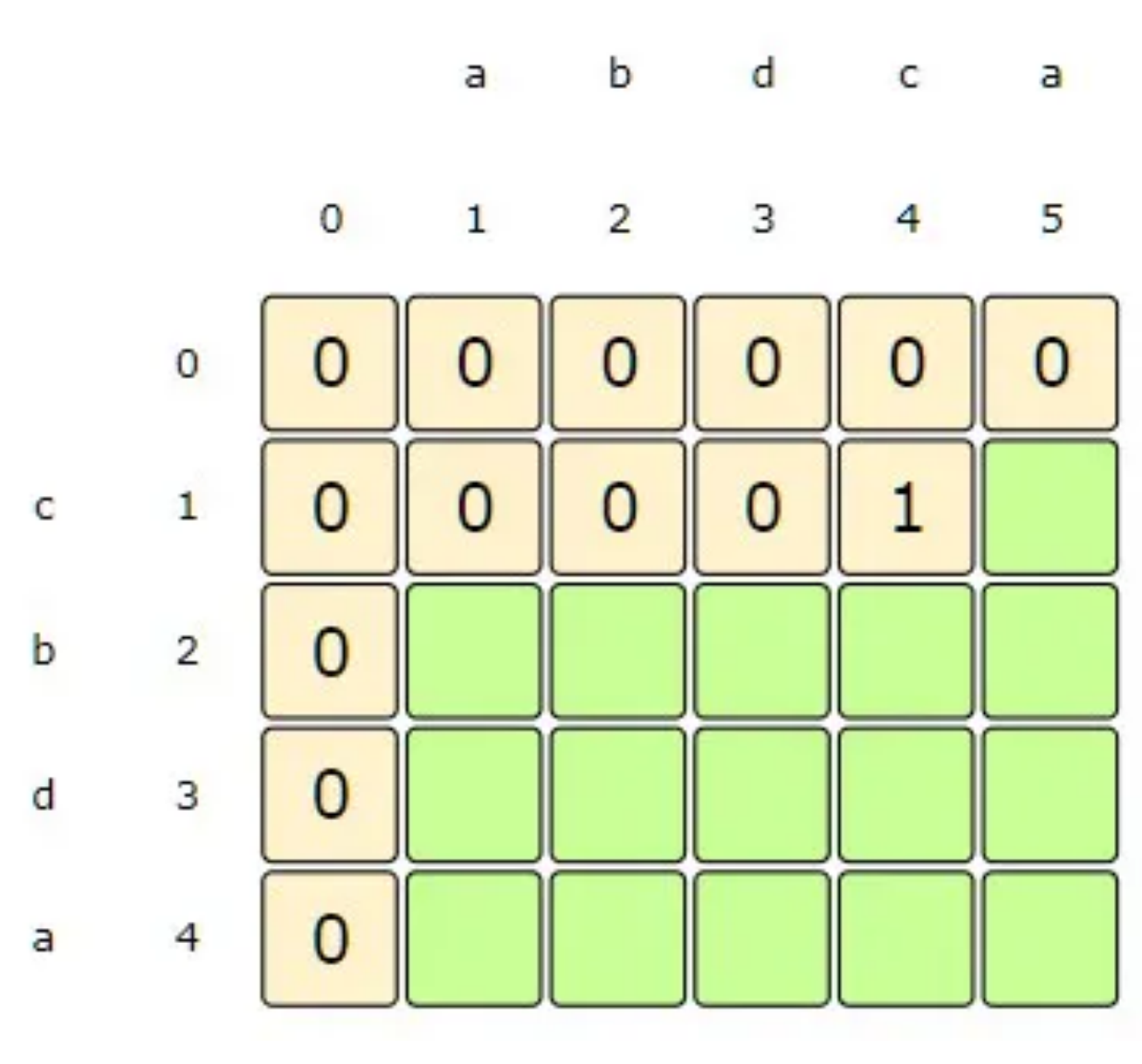
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0				
b	2	0					
d	3	0					
a	4	0					

i:1, j:1 => dp[i][j] = 0, as s1[i] != s2[j]



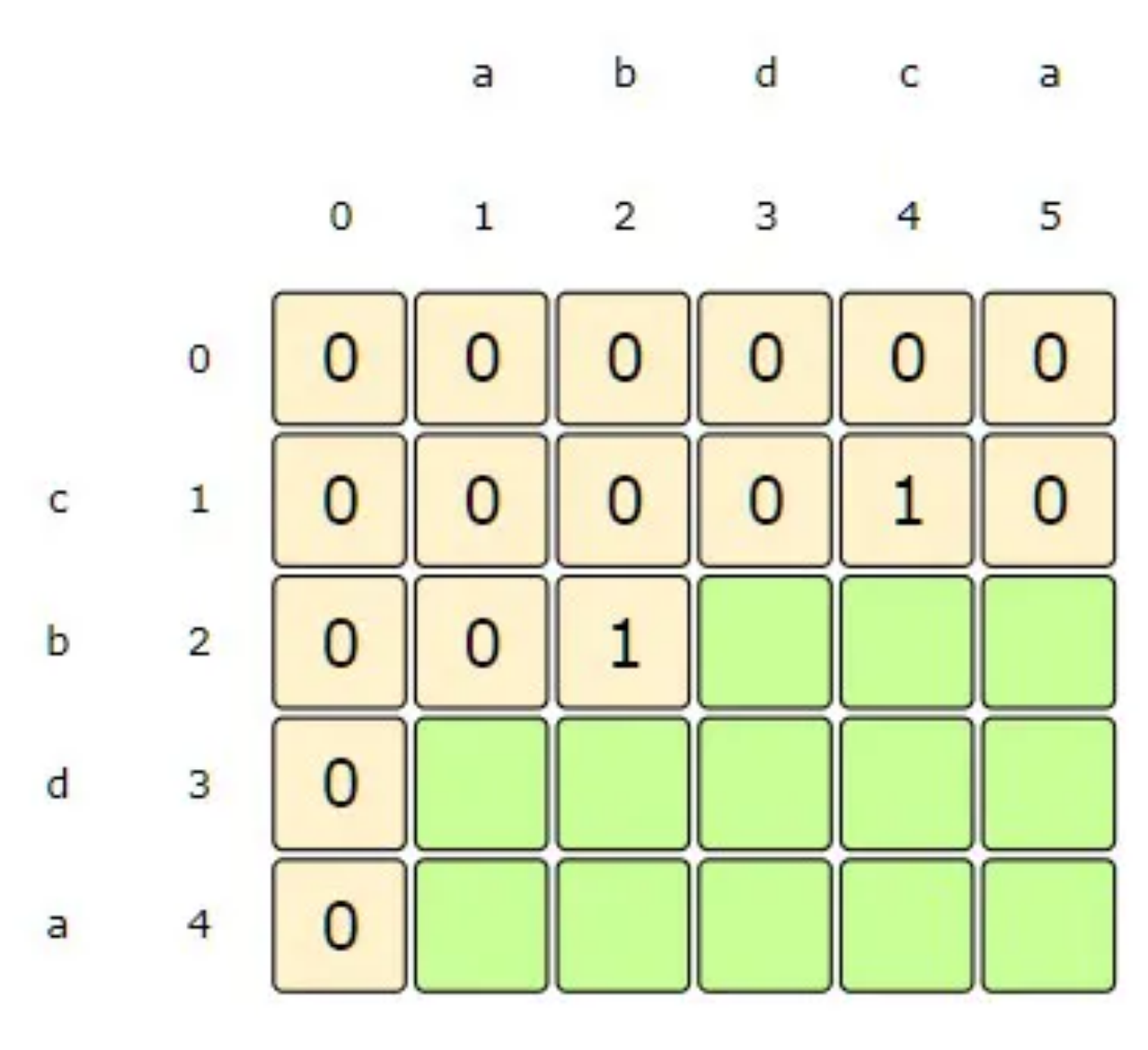
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0			
b	2	0					
d	3	0					
a	4	0					

i:1, j:2 => dp[i][j] = 0, as s1[i] != s2[j]



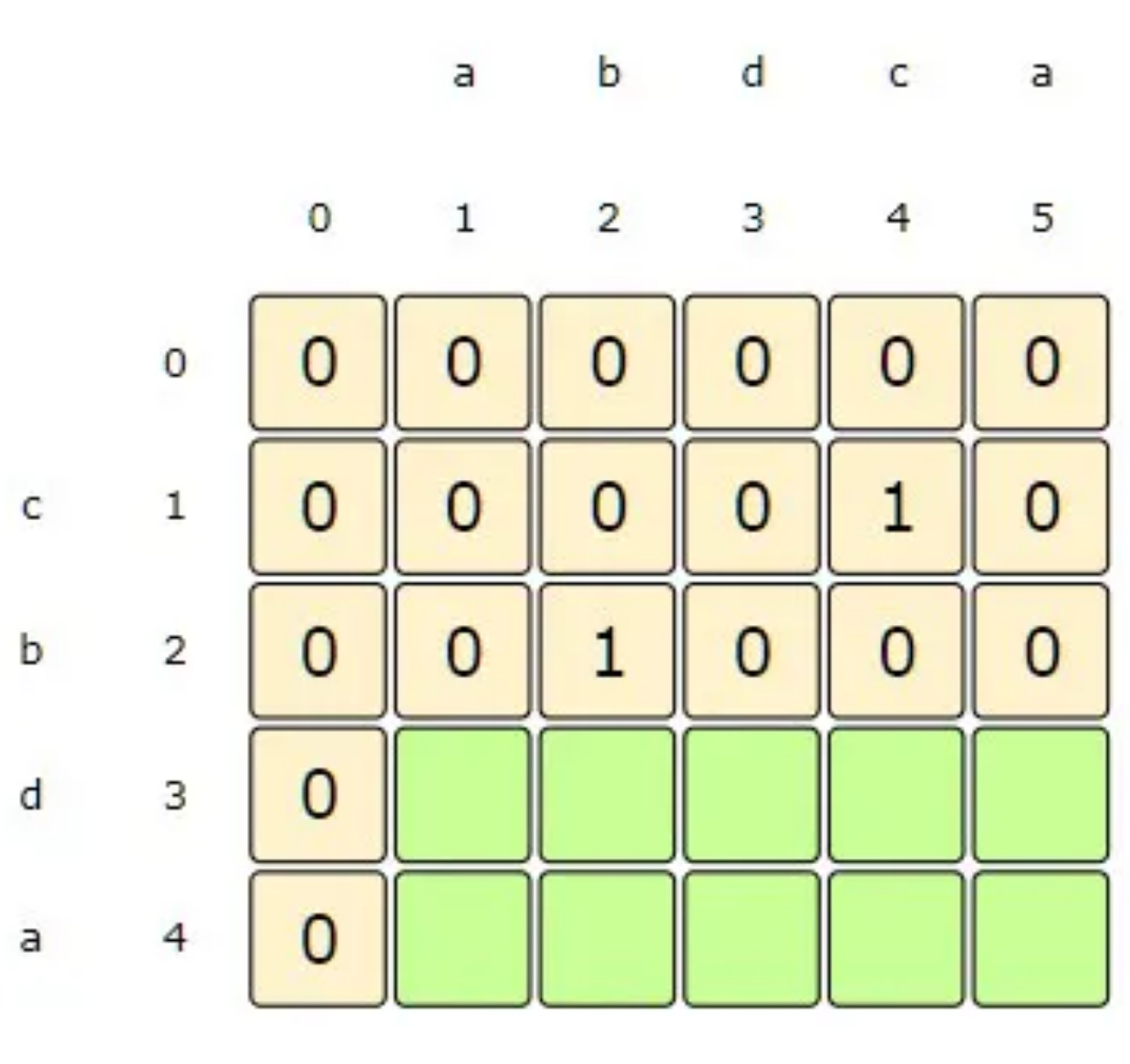
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	
b	2	0					
d	3	0					
a	4	0					

i:1, j:4 => dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]



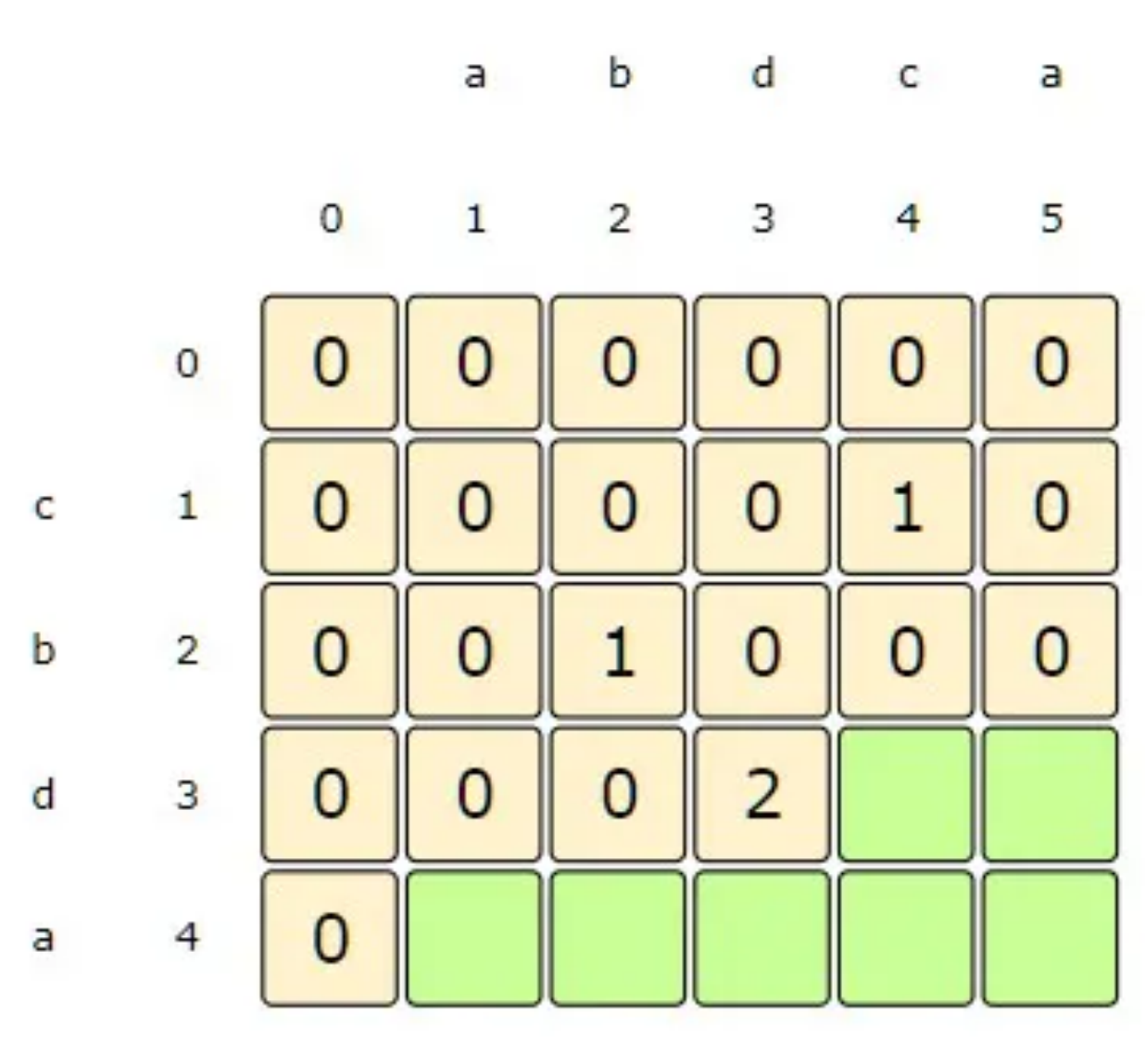
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	0
b	2	0	0	1			
d	3	0					
a	4	0					

i:2, j:2=> dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]



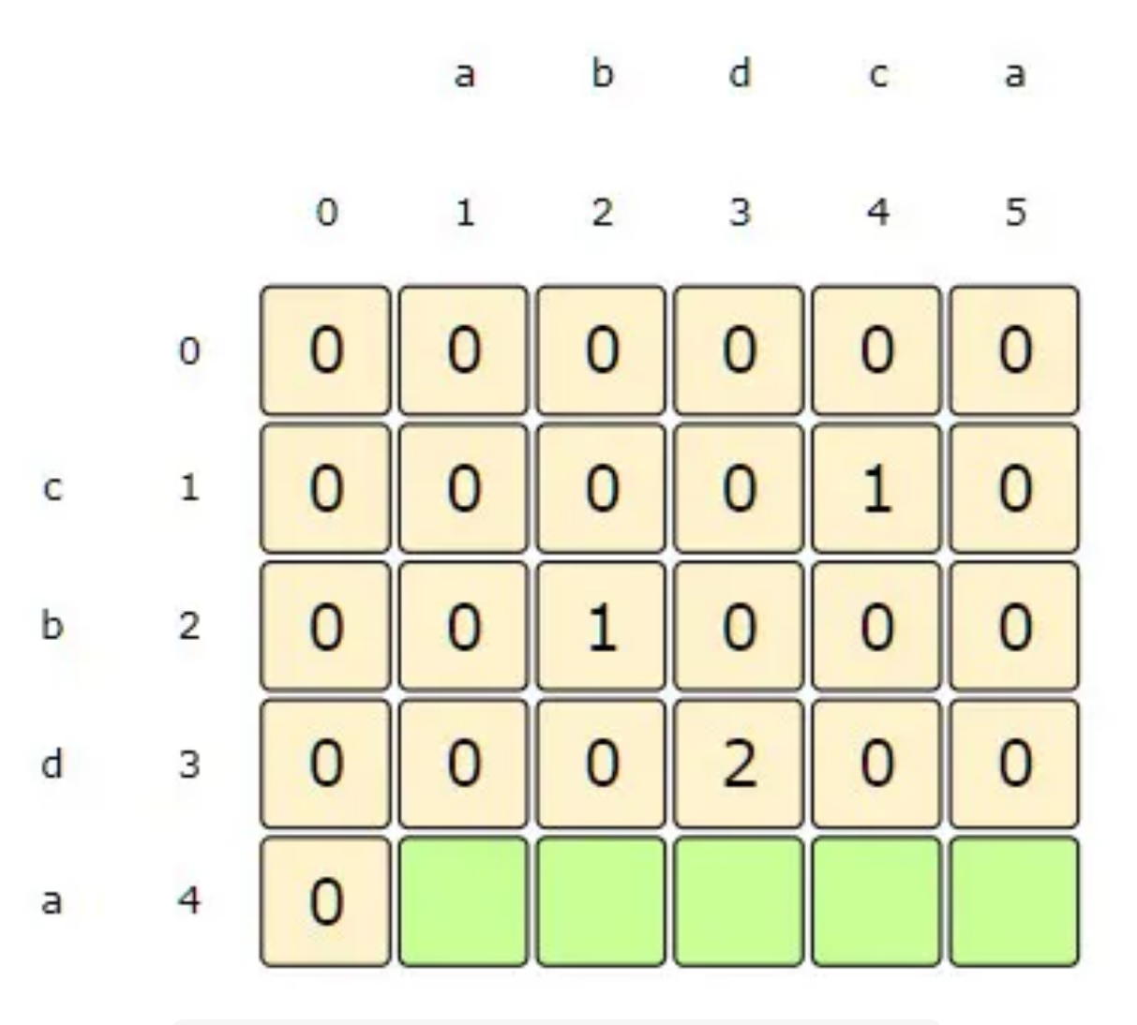
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	0
b	2	0	0	1	0	0	0
d	3	0					
a	4	0					

i:2, j:3-5 => dp[i][j] = 0, as s1[i] != s2[j]



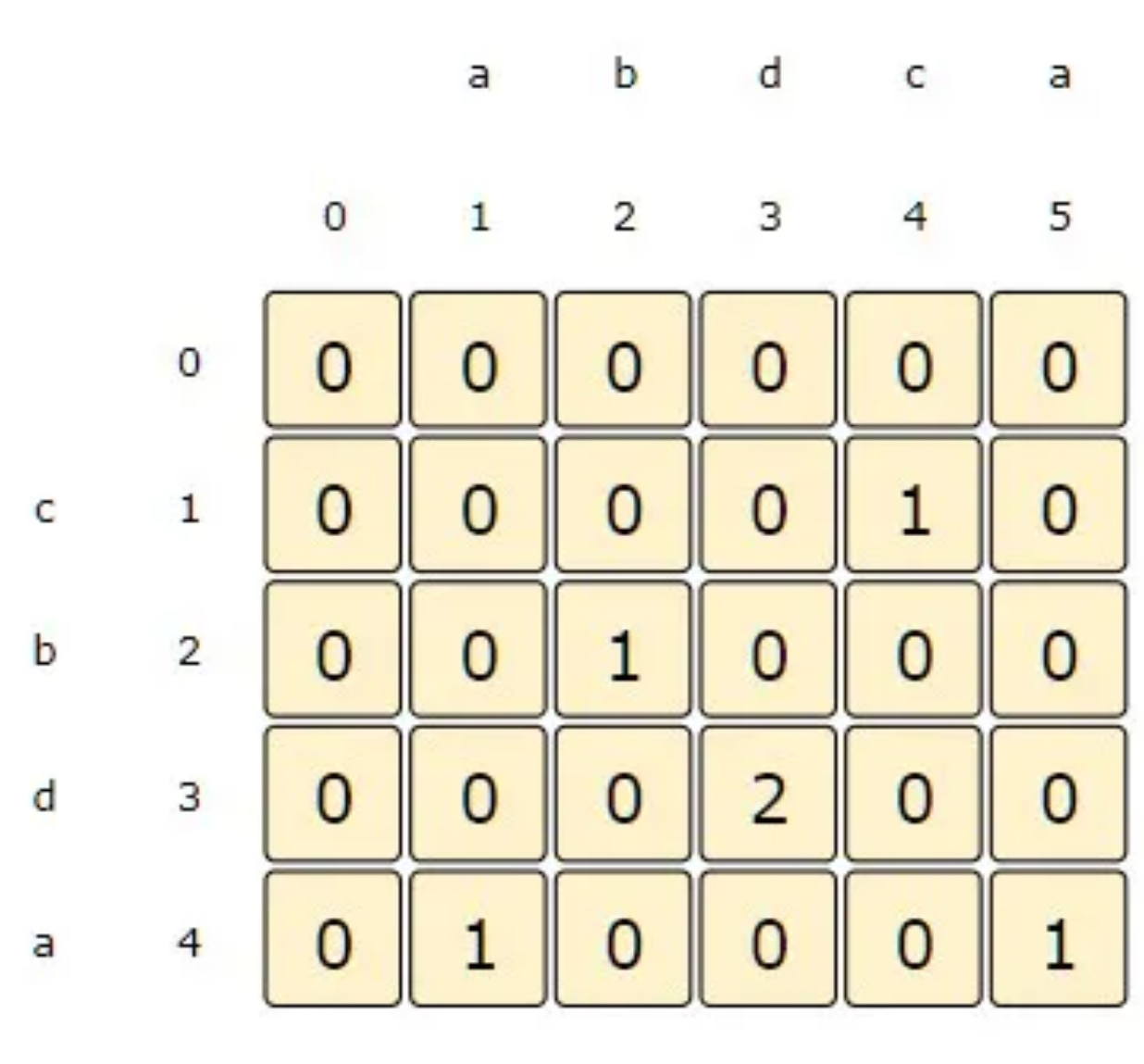
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	0
b	2	0	0	1	0	0	0
d	3	0	0	0	2		
a	4	0					

i:3, j:3=> dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]



			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	0
b	2	0	0	1	0	0	0
d	3	0	0	0	2	0	0
a	4	0					

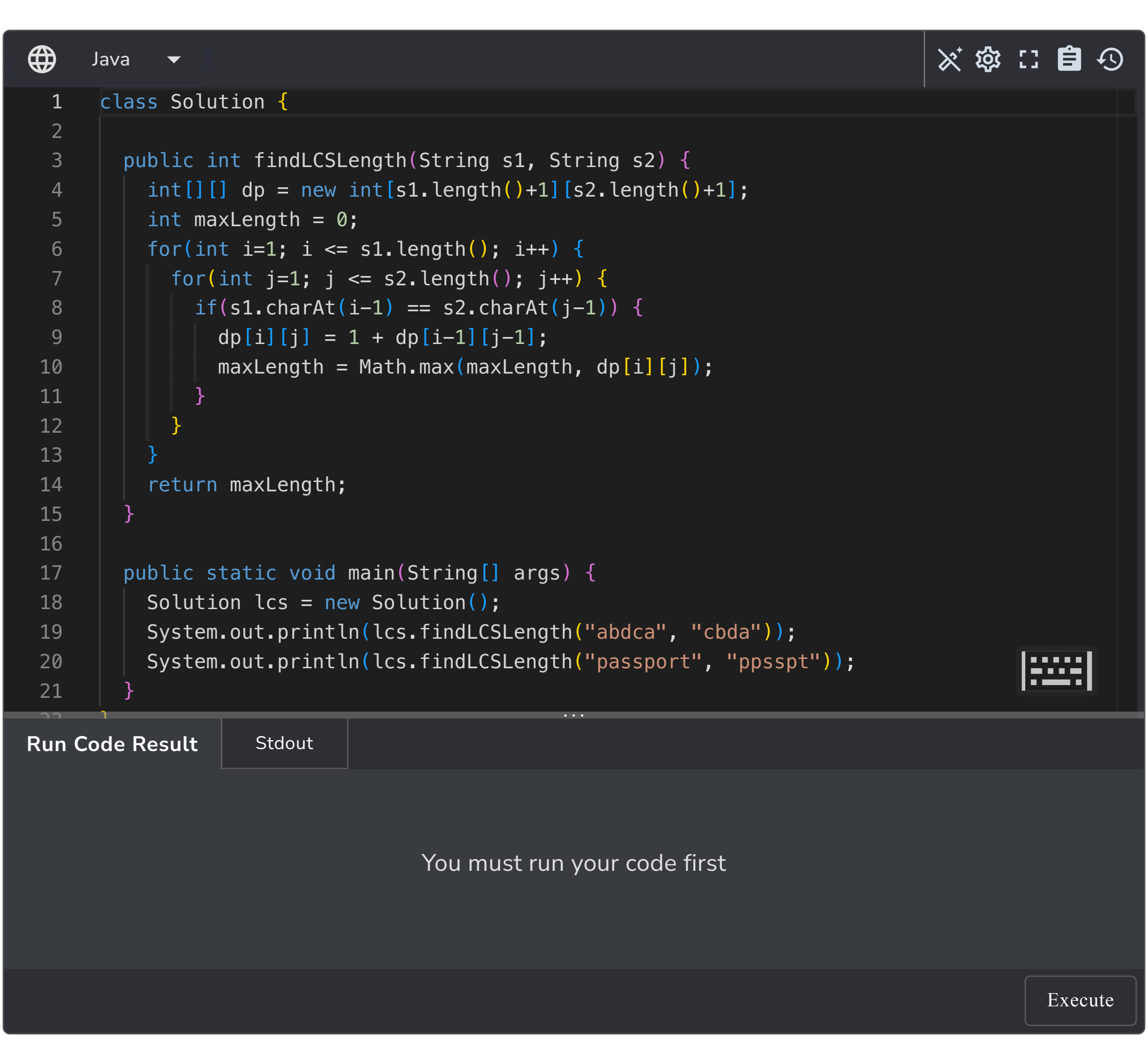
i:3, j:4-5=> dp[i][j] = 0, as s1[i] != s2[j]



			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	0
b	2	0	0	1	0	0	0
d	3	0	0	0	2	0	0
a	4	0	1	0	0	0	1

i:4, j:5=> dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]

From the above visualization, we can clearly see that the longest common substring is of length '2'-- as shown by `dp[3][3]`. Here is the code for our bottom-up dynamic programming approach:



```java
class Solution {

  public int findLCSLength(String s1, String s2) {
    int[][] dp = new int[s1.length()+1][s2.length()+1];
    int maxLength = 0;
    for(int i=1; i <= s1.length(); i++) {
      for(int j=1; j <= s2.length(); j++) {
        if(s1.charAt(i-1) == s2.charAt(j-1)) {
          dp[i][j] = 1 + dp[i-1][j-1];
          maxLength = Math.max(maxLength, dp[i][j]);
        }
      }
    }
    return maxLength;
  }

  public static void main(String[] args) {
    Solution lcs = new Solution();
    System.out.println(lcs.findLCSLength("abdca", "cbda"));
    System.out.println(lcs.findLCSLength("passport", "ppsspt"));
  }
```

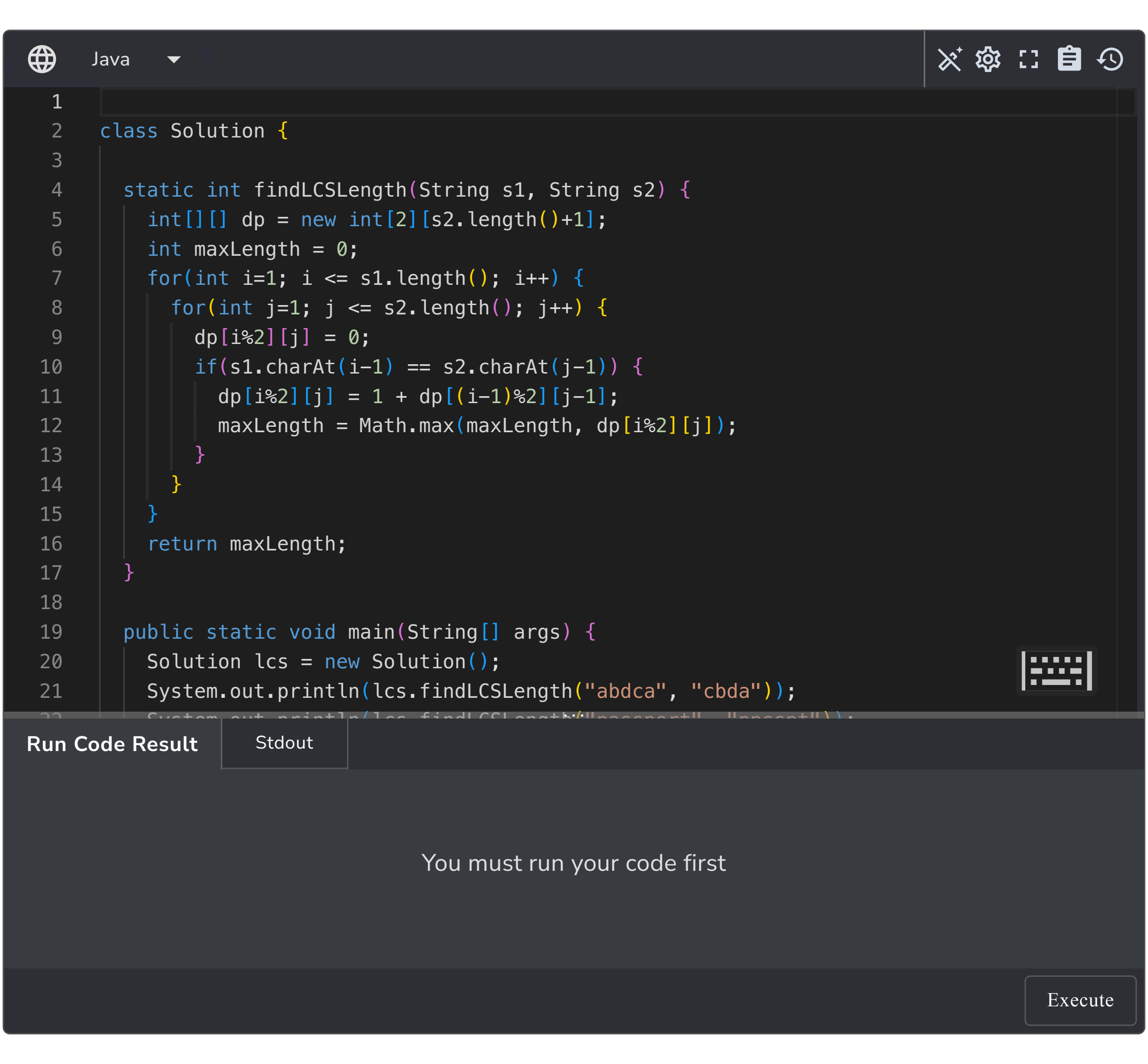
The time and space complexity of the above algorithm is $O(m*n)$, where 'm' and 'n' are the lengths of the two input strings.

Challenge

Can we further improve our bottom-up DP solution? Can you find an algorithm that has $O(n)$ space complexity?

Hint

We only need one previous row to find the optimal solution!



```java

class Solution {

  static int findLCSLength(String s1, String s2) {
    int[][] dp = new int[2][s2.length()+1];
    int maxLength = 0;
    for(int i=1; i <= s1.length(); i++) {
      for(int j=1; j <= s2.length(); j++) {
        dp[i%2][j] = 0;
        if(s1.charAt(i-1) == s2.charAt(j-1)) {
          dp[i%2][j] = 1 + dp[(i-1)%2][j-1];
          maxLength = Math.max(maxLength, dp[i%2][j]);
        }
      }
    }
    return maxLength;
  }

  public static void main(String[] args) {
    Solution lcs = new Solution();
    System.out.println(lcs.findLCSLength("abdca", "cbda"));
```

← Previous
Longest Common Substring

Next →
Longest Common Subsequence

☐ Mark as Completed